

DreamCoder and Wake-Sleep Library Learning

MAT2320-2026 – Mircea Petrache

Abstract

These lecture notes present *DreamCoder*, a Bayesian program synthesis system introduced by Ellis et al. (2021) that jointly learns a growing library of program primitives and a neural network to guide the search for programs. We develop the framework in three layers: the probabilistic formulation (library as latent variable, programs as samples from a probabilistic context-free grammar), the wake-sleep algorithm that performs approximate Bayesian inference, and the connection of both to the Helmholtz machine and to modern variational inference. We then survey applications of DreamCoder-style systems to the Abstraction and Reasoning Corpus (ARC-AGI), where the approach is conceptually closest to what the benchmark was designed to measure. Throughout, we emphasise the mathematical content and the relationship to the Kolmogorov / MDL view of intelligence introduced in earlier lectures.

Contents

1	Motivation and Problem Setting	3
1.1	Why program synthesis?	3
1.2	Inductive program synthesis	3
1.3	The MDL foundation	4
2	Probabilistic Formulation	4
2.1	Generative model	4
2.2	The inference goal	5
2.3	The recognition model	5
3	The Wake-Sleep Algorithm	6
3.1	Wake phase: program search	6
3.2	Sleep / Abstraction phase: library growth	6
3.3	Sleep / Dreaming phase: training the recognition model	7
4	The Library as a Probabilistic Grammar	8
4.1	Typed grammars	8
4.2	Probabilities update with the library	8
4.3	Abstraction gain, formally	8
4.4	Connection to compositional generalisation	9
5	Connection to the Helmholtz Machine and VAEs	9
5.1	Variational inference primer	9
5.2	DreamCoder as a discrete, hierarchical wake-sleep system	9
6	Experimental Results	10
6.1	Domains evaluated	10
6.2	Ablations	10

6.3	Kepler’s third law: a case study	11
7	Applications to ARC-AGI	11
7.1	Why DreamCoder is conceptually aligned with ARC	11
7.2	PeARL and the Bober-Irizar et al. system	11
7.3	Hybrid LLM + DreamCoder approaches	12
7.4	The fast-versus-smart tension	12
8	Connections to Course Themes	12
9	Limitations and Open Problems	12
9.1	Limitations	12
9.2	Open problems	13
10	Conclusion	14

1 Motivation and Problem Setting

1.1 Why program synthesis?

The dominant paradigm in machine learning over the past decade has been to train large parametric models on large datasets within a fixed task domain. Such models generalise well to held-out examples from the same distribution, but they struggle on tasks that require composing previously learned concepts in novel ways, that demand sample-efficient learning from a handful of examples, or that require interpretable solutions (Lake and Baroni, 2018; Chollet, 2019).

Human experts behave differently. A mathematician who has internalised the notions of *group*, *ring*, and *module* can read a research paper in algebra without explicit re-instruction on those concepts; she recognises them, combines them, and extends them. The history of mathematics itself shows the same pattern at a larger scale: each generation extracts a small number of new abstractions from the work of the previous one—limit, derivative, manifold, sheaf—and the new abstractions enable problems that were previously intractable. Tenenbaum et al. (2011) argued that this acquisition of *conceptual vocabulary* is the central engine of human intelligence, and that it cannot be reduced to pattern recognition on raw inputs.

The thesis of Ellis et al. (2021)—inherited from a longer tradition of Bayesian program learning (Lake et al., 2015)—is that expert problem-solving is driven by powerful *languages for thinking* about problems and their solutions. Acquiring expertise therefore means learning those languages, in addition to learning to use them. DreamCoder operationalises this thesis: it is a system that, given a set of tasks, jointly infers (i) a library of reusable program primitives and (ii) solutions to each task expressed in that library.

1.2 Inductive program synthesis

The technical problem DreamCoder addresses is *inductive program synthesis*: given a set of tasks $\mathcal{X} = \{x_1, \dots, x_n\}$, each specified by a small number of input-output examples, find programs $\{\varrho_{x_i}\}$ that satisfy all examples of each task. This problem is at least as old as automatic programming (Gulwani, 2011) and has three intrinsic challenges:

- (i) **Combinatorial explosion.** The number of syntactically valid programs grows exponentially with program length. A naive enumeration becomes intractable beyond a few primitives composed at depth four or five.
- (ii) **Generalisation, not interpolation.** A program is correct only if it produces the right output for *every* possible input, not only the few inputs shown as examples. This rules out approaches that merely memorise the training set.
- (iii) **Interpretability and reuse.** A useful solution should be a program that a human can read, understand, and modify; and that the learner can call upon to solve future tasks.

Prior work attacks (i) by restricting attention to a hand-crafted *Domain-Specific Language* (DSL) of primitives. Within a well-designed DSL, exhaustive search becomes feasible and the resulting programs are interpretable by construction. The cost is that all relevant concepts must be specified upfront by the human designer; the system cannot grow its vocabulary in response to the task distribution.

DreamCoder’s departure is to treat the DSL itself as a latent variable to be inferred from data. The system starts with a small set of generic primitives and grows the DSL through a Bayesian learning procedure. The resulting language captures the structure of the task distribution and dramatically reduces the effective search space for future problems.

1.3 The MDL foundation

The criterion for what makes a good library is provided by the *Minimum Description Length (MDL) principle* (Rissanen, 1978; Li and Vitányi, 2008), which in turn is the practical face of Kolmogorov complexity introduced in earlier lectures. For a library L and programs $\{\varrho_x\}$ that solve tasks $\mathcal{X}$, the joint description length is:

$$\text{DL}(L, \{\varrho_x\}, \mathcal{X}) = \underbrace{|L|}_{\text{library cost}} + \sum_{x \in \mathcal{X}} \underbrace{|\varrho_x|_L}_{\text{program length encoded under } L}, \quad (1)$$

where $|L|$ is the cost of specifying L in bits and $|\varrho_x|_L$ is the length of ϱ_x when its tokens are drawn from L . The MDL objective is to find $(L^*, \{\varrho_x^*\})$ that minimises (1), subject to the constraint that each ϱ_x solves task x .

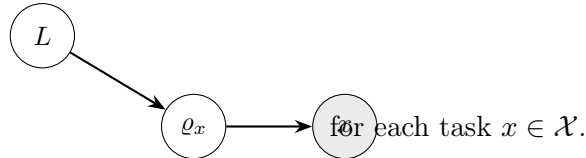
The intuition is that a useful library is one that compresses the solutions across the task distribution. Adding an entry to L costs bits, but if that entry appears in many programs ϱ_x , the savings on $|\varrho_x|_L$ across all tasks more than offset the cost. This makes MDL a principled criterion for abstraction: a primitive joins the library if and only if its use is amortised over enough tasks.

Remark 1.1. Equation (1) is precisely the objective that an *ideal* Kolmogorov-style learner would minimise. Because Kolmogorov complexity is uncomputable (Theorem 2.2 of the main lecture notes), MDL is in general also intractable; DreamCoder’s contribution is an algorithmic *approximation* that performs well in practice.

2 Probabilistic Formulation

2.1 Generative model

DreamCoder casts the MDL objective in Bayesian form. The graphical model has the following structure:



The components are:

- L : the *library*, a typed DSL whose entries are programs together with a probability distribution over them.
- ϱ_x : a program for task x , drawn from a prior $P(\varrho \mid L)$ induced by the library.
- x : the observed task, consisting of input-output examples.

The joint distribution factorises as:

$$P(L, \{\varrho_x\}, \mathcal{X}) = P(L) \prod_{x \in \mathcal{X}} P(\varrho_x \mid L) P(x \mid \varrho_x). \quad (2)$$

The two task-level factors have explicit forms:

The program prior $P(\varrho \mid L)$. The library L defines a *probabilistic context-free grammar (PCFG)* over typed expression trees. Each library entry (f_i, τ_i, p_i) specifies a named function f_i , its type signature τ_i , and its production probability p_i . For a program ϱ , the prior is:

$$P(\varrho \mid L) = \prod_{v \in \text{nodes}(\varrho)} p_{f(v)}, \quad (3)$$

where $f(v)$ is the library entry used at node v . Equivalently, the *description length* under L is $|\varrho|_L = -\sum_v \log_2 p_{f(v)}$ bits, which recovers the MDL view: more probable programs are shorter.

The likelihood $P(x \mid \varrho)$. For deterministic tasks (the typical setting in DreamCoder),

$$P(x \mid \varrho) = \mathbf{1}[\varrho \text{ produces the correct output on every example of } x]. \quad (4)$$

A program either solves a task or it does not. There is no partial credit at this stage; the soft scoring lives entirely in the prior (3).

2.2 The inference goal

DreamCoder’s central inference problem is to find $(L, \{\varrho_x\})$ maximising the joint posterior:

$$\arg \max_{L, \{\varrho_x\}} P(L) \prod_{x \in \mathcal{X}} P(\varrho_x \mid L) P(x \mid \varrho_x). \quad (5)$$

By (4), the second factor is either 1 or 0, so the arg max is taken over $(L, \{\varrho_x\})$ such that every ϱ_x solves its task. Taking logarithms and assuming a uniform prior on libraries (an assumption refined later), this becomes:

$$\arg \max_{L, \{\varrho_x\}} \sum_{x \in \mathcal{X}} \log P(\varrho_x \mid L) = \arg \min_{L, \{\varrho_x\}} \sum_{x \in \mathcal{X}} |\varrho_x|_L \text{ (subject to correctness),} \quad (6)$$

which is exactly the MDL objective (1) up to the library cost $|L|$. The library cost is incorporated through a structured prior on L penalising long entries (Section 3.2).

2.3 The recognition model

Exact maximisation of (5) is intractable because the space of programs is exponentially large. DreamCoder introduces an *amortised recognition model* $Q_\theta(\varrho \mid x)$, a neural network parameterised by θ , that approximates the posterior $P(\varrho \mid x, L) \propto P(x \mid \varrho)P(\varrho \mid L)$. The role of Q is to predict, given the examples of a task, which programs are likely to solve it—thereby guiding the program search.

This is the same idea as the inference network in a variational autoencoder (Kingma and Welling, 2014) or the recognition network in the Helmholtz machine (Dayan et al., 1995): the model is trained to invert the generative process *on average*, so that future inference is fast.

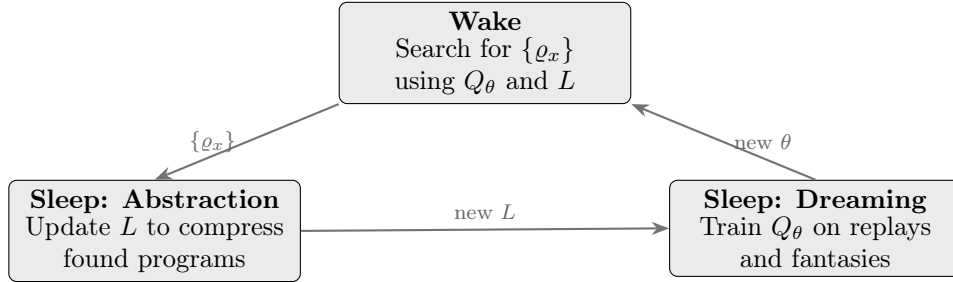
The architecture has three stages:

- (i) **Example encoder.** A domain-specific neural module embeds each input-output example into a fixed-size vector. For list-processing tasks, this is an LSTM. For image-based tasks (e.g. Logo graphics), it is a convolutional network. For grid tasks (e.g. ARC), it is a small transformer over discretised pixels.
- (ii) **Task pooler.** The per-example embeddings are pooled (typically by mean or attention) into a single task embedding e_x .
- (iii) **Program decoder.** A standard autoregressive decoder generates the program token-by-token, where each token is a library entry, conditioned on e_x and the partial program so far.

The training of Q_θ is performed in the dreaming phase, described in Section 3.3.

3 The Wake-Sleep Algorithm

DreamCoder performs approximate Bayesian inference on (5) through an iterative procedure that alternates three phases. The structure is illustrated below:



Each iteration updates one component while holding the others fixed: **Wake** updates $\{\varrho_x\}$ holding L and θ fixed; **Abstraction** updates L holding $\{\varrho_x\}$ fixed; **Dreaming** updates θ holding L fixed. This three-block coordinate-descent structure is what allows the otherwise intractable joint inference to make progress (Hinton et al., 1995).

3.1 Wake phase: program search

The wake phase fixes L and θ and searches, for each task x , for the highest-scoring program that solves the task. The scoring function combines the recognition model and the prior:

$$\text{score}(\varrho, x) = \log Q_\theta(\varrho | x) + \log P(\varrho | L) = \log Q_\theta(\varrho | x) - |\varrho|_L. \quad (7)$$

Programs that are both likely under the neural model *and* short under the library prior are explored first.

Search procedure. The standard implementation uses *best-first beam search* guided by (7), with each candidate executed against the task’s input-output examples. Two crucial properties of this procedure:

- (a) **Execution is a hard correctness oracle.** A candidate program is accepted only if it produces the correct output on every example. This eliminates an entire class of failure modes that plague test-time fine-tuning approaches, where the model can be confidently wrong.
- (b) **Search is anytime.** A budget (typically wall-clock time) limits exploration; any program found within the budget is kept. Tasks that cannot be solved within the budget remain unsolved until a later iteration when L or Q_θ has improved.

The wake phase outputs, for each task x , the best correct program $\varrho_x^* = \arg \max_{\varrho \text{ correct}} \text{score}(\varrho, x)$, or $\perp$ if no correct program is found within the budget.

3.2 Sleep / Abstraction phase: library growth

The abstraction phase fixes $\{\varrho_x^*\}$ and seeks a library L^* that maximises (6), augmented by a structural prior on L :

$$L^* = \arg \max_L \left[\log P(L) + \sum_{x \in \mathcal{X}} \log P(\varrho_x^* | L) \right], \quad (8)$$

where $\log P(L) = -\beta \cdot \text{size}(L)$ is a regulariser that penalises libraries with many or long entries (a hyperparameter β controls the trade-off). The objective (8) is exactly the MDL trade-off: extending L pays for itself only if the compression gain on the programs exceeds the size cost.

Refactoring. Optimising (8) exactly is intractable: the space of candidate library extensions is the set of all expressions in the combined language, modulo α -equivalence and β -reduction. DreamCoder uses a refactoring algorithm based on *version spaces* and *E-graph* equality saturation (Willsey et al., 2021) to enumerate semantically equivalent rewrites of each program and identify sub-expressions that appear in many of them.

The algorithm proceeds greedily: at each step, it considers all candidate sub-expressions g that appear in the current refactoring of $\{\varrho_x^*\}$, computes the *compression gain*

$$\Delta_g = \underbrace{|g|}_{\text{cost of adding } g \text{ to } L} - \sum_{x: g \text{ used in } \varrho_x^*} \left[|\varrho_x^*|_L - |\varrho_x^*[g]|_{L \cup \{g\}} \right], \quad (9)$$

where $\varrho_x^*[g]$ denotes the program ϱ_x^* rewritten with the new abstraction g . The abstraction with the most negative Δ_g is added to L , and the procedure repeats until no candidate yields $\Delta_g < 0$.

Example 3.1 (Discovering `map`). Suppose two tasks have been solved during the wake phase:

$$\begin{aligned} \varrho_1 &= \mathbf{Y}(\lambda r l. \text{if nil?}(l) \text{ nil cons}(+(\text{car } l)(\text{car } l)) (r(\text{cdr } l))), \\ \varrho_2 &= \mathbf{Y}(\lambda r l. \text{if nil?}(l) \text{ nil cons}(-(\text{car } l) 1) (r(\text{cdr } l))). \end{aligned}$$

The two programs share the skeleton of a list-traversal recursion. Refactoring extracts:

$$\mathbf{map} := \lambda f. \mathbf{Y}(\lambda r l. \text{if nil?}(l) \text{ nil cons}(f(\text{car } l)) (r(\text{cdr } l))),$$

which, when added to L , allows ϱ_1 and ϱ_2 to be rewritten as $\mathbf{map}(\lambda z. +(z z))$ and $\mathbf{map}(\lambda z. -(z 1))$ respectively. The compression gain $-\Delta_{\mathbf{map}}$ is positive whenever the saving across programs that use `map` exceeds its addition cost. After this step, future tasks involving element-wise list transformations will be found with much shorter programs.

3.3 Sleep / Dreaming phase: training the recognition model

The dreaming phase fixes L and trains Q_θ to better predict the posterior $P(\varrho \mid x, L)$. The training objective is:

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{(\varrho, x) \sim P_{\text{train}}} [\log Q_\theta(\varrho \mid x)], \quad (10)$$

where P_{train} is a mixture of two sources:

- **Replays.** Pairs (x, ϱ_x^*) where ϱ_x^* is a program found during a recent wake phase. Replays ensure Q_θ does not forget how to solve tasks that have already been solved.
- **Fantasies.** Pairs (x, ϱ) where $\varrho \sim P(\varrho \mid L)$ is sampled from the library prior and x is the task induced by executing ϱ on random inputs. Fantasies provide an essentially unlimited supply of synthetic training data and let Q_θ generalise beyond the small set of training tasks.

The role of fantasies. DreamCoder is typically run with 100–200 training tasks per domain—far too few to train a high-capacity recognition network from scratch. Fantasies solve this data scarcity by exploiting the generative model: given the current library, one can sample as many synthetic (program, task) pairs as desired. Since the generative process is itself the system’s best model of the domain, training on fantasies is equivalent to training on the model’s own hypotheses about what plausible tasks and solutions look like. This is the deep reason for the “dreaming” metaphor: the system rehearses imagined scenarios consistent with its current world model.

Loss function. For a program $\varrho = (t_1, t_2, \dots, t_m)$ generated token by token, the recognition model factorises as $Q_\theta(\varrho \mid x) = \prod_{i=1}^m Q_\theta(t_i \mid t_{<i}, x)$, and (10) becomes a standard autoregressive cross-entropy loss. Training proceeds by stochastic gradient descent over mini-batches of (replay, fantasy) pairs in a fixed ratio (typically 1:1).

4 The Library as a Probabilistic Grammar

We now examine the structure of the library L more carefully. Understanding L as a probabilistic typed grammar is what gives (3) its operational meaning and what makes the abstraction phase well-posed.

4.1 Typed grammars

A library $L = \{(f_i, \tau_i, p_i)\}_{i=1}^{|L|}$ consists of:

- f_i : a named function, possibly defined as a λ -expression in the language of L *excluding* f_i itself (libraries are always definitionally well-founded).
- τ_i : a polymorphic type signature, e.g., $\text{map} : (\alpha \rightarrow \beta) \rightarrow \text{list}(\alpha) \rightarrow \text{list}(\beta)$.
- $p_i \in [0, 1]$: a production probability, with the constraint that for every type τ , the probabilities of entries whose return type matches τ sum to 1.

A program ϱ of type τ is an expression tree where each node is a library entry and the types match according to the standard rules of typed λ -calculus. The probability $P(\varrho \mid L)$ of generating ϱ by type-conditional sampling is the product in (3).

Remark 4.1. Restricting library entries to be typed is not a minor technicality. It guarantees that the generative process (3) only samples well-typed programs, which dramatically reduces the effective search space. For example, an integer cannot be applied as a function, a list-returning expression cannot be passed where a boolean is expected, and so on.

4.2 Probabilities update with the library

Each iteration of DreamCoder updates not only the set of library entries but also the production probabilities $\{p_i\}$, by maximum-likelihood estimation on the programs found in the most recent wake phase. Concretely:

$$p_i^{\text{new}} \propto \sum_{x \in \mathcal{X}} \#\{\text{uses of } f_i \text{ in } \varrho_x^*\}, \quad (11)$$

normalised within each return-type class. Frequently used entries become more probable, which biases the program prior (3) toward combinations that have proved useful in the past—a form of *learned inductive bias*.

4.3 Abstraction gain, formally

Putting the pieces together, the compression gain (9) of adding a candidate abstraction g to the library has a clean mathematical interpretation:

$$-\Delta_g = \underbrace{\sum_{x: g \in \varrho_x^*} [|\varrho_x^*|_L - |\varrho_x^*[g]|_{L \cup \{g\}}]}_{\text{bits saved by rewriting programs that use } g} - \underbrace{|g|}_{\text{bits to define } g}. \quad (12)$$

This is the standard MDL trade-off in a structured form. The first term grows linearly with the number of programs that use g ; the second is constant. Hence an abstraction joins the library if and only if it is used across enough programs.

4.4 Connection to compositional generalisation

The library structure is what makes DreamCoder *compositional* in the sense of Definition 4.3 of the main lecture notes (Lake and Baroni, 2018; Hupkes et al., 2020). A library entry f_i is a primitive in the sense that, once defined, it can be combined with other primitives in any well-typed way. New tasks are solved by composing existing primitives, and the system’s behaviour on a novel combination is deterministic and predictable from its behaviour on the parts.

This is the operational reason DreamCoder is conceptually well-suited to benchmarks that measure compositional generalisation: the model literally solves new tasks by recombining the parts it has learned, not by interpolating in a continuous representation space.

5 Connection to the Helmholtz Machine and VAEs

DreamCoder’s three-phase loop is a structured variant of the wake-sleep algorithm introduced by Hinton et al. (1995) for unsupervised neural networks, refined into the Helmholtz machine (Dayan et al., 1995), and generalised into modern variational inference and the variational autoencoder (Kingma and Welling, 2014). Understanding this connection clarifies which parts of DreamCoder are standard and which are novel.

5.1 Variational inference primer

In generic variational inference, one has observations x , latent variables z , a generative model $P_\phi(x, z)$, and an approximate posterior $Q_\theta(z | x)$. The goal is to maximise the evidence lower bound (ELBO):

$$\text{ELBO}(\theta, \phi; x) = \mathbb{E}_{Q_\theta(z|x)}[\log P_\phi(x | z)] - \text{KL}(Q_\theta(z | x) \| P_\phi(z)) \leq \log P_\phi(x). \quad (13)$$

The wake-sleep algorithm alternately updates ϕ (wake) and θ (sleep). In the original Helmholtz machine, both updates are by gradient ascent on (13). In the variational autoencoder, the two updates are merged via the reparameterisation trick, but the two-direction structure of inference remains.

5.2 DreamCoder as a discrete, hierarchical wake-sleep system

The mapping between DreamCoder and the standard wake-sleep / VAE framework is summarised in Table 1.

Component	Helmholtz / VAE	DreamCoder
Latent variable	Continuous $z \in \mathbb{R}^d$	Discrete program ϱ
Likelihood	$P_\phi(x z)$ (Gaussian or Bernoulli)	$\mathbf{1}[\varrho \text{ solves } x]$
Prior	Fixed $\mathcal{N}(0, I)$ on z	Learnable library L on programs
Recognition model	$Q_\theta(z x)$ (Gaussian)	$Q_\theta(\varrho x)$ (autoregressive over tokens)
Wake update	Decoder gradient	Discrete program search via beam search
Sleep update	Encoder gradient	(a) Abstraction (b) Dreaming
Library / Prior update	— (prior is fixed)	MDL-driven refactoring of L

Table 1: Correspondence between Helmholtz / VAE inference and DreamCoder.

Three differences are worth emphasising:

- (a) **Discrete latent.** The latent variable is a program, not a continuous vector. There is no reparameterisation trick; the wake phase performs exact discrete search.

- (b) **Hierarchical prior.** The prior on programs is itself a latent variable (the library) that is updated across iterations. This is a key structural difference: in a VAE the prior is fixed, in DreamCoder the prior is learned.
- (c) **Hard likelihood.** The likelihood is binary (correct or not), which makes inference exact rather than amortised at this step.

These differences are not modifications of the algorithm; they are forced by the problem domain. Programs cannot be treated as continuous vectors, and a correctness criterion is inherently discrete.

6 Experimental Results

Ellis et al. (2021) evaluate DreamCoder on eight heterogeneous domains. The results are best understood not by absolute accuracy numbers (which depend on time budget and primitive choice) but by what the system *discovers*.

6.1 Domains evaluated

- **List processing.** 218 tasks involving lists of integers (sort, filter, sum-of-squares, etc.). DreamCoder rediscovers the higher-order functions `map`, `filter`, `fold`, `zip`, `unique`, and `sort`, in that order, across six wake-sleep iterations.
- **Text editing.** 109 tasks from the FlashFill suite (Gulwani, 2011). DreamCoder matches FlashFill’s accuracy despite starting from generic string primitives, with the library eventually containing FlashFill-style concepts such as `split-on-separator` and `capitalise-first-letter`.
- **Logo graphics.** 160 turtle-graphics drawing tasks (squares, spirals, snowflakes). The system discovers subroutines for `polygon`, `n-gon-side`, and recursive fractal construction.
- **Symbolic regression.** Recovery of physical laws from data, including Kepler’s third law $T^2 \propto r^3$, Coulomb’s law, and Snell’s law. The library grows to include $(\cdot)^2$, $(\cdot)^3$, and rational exponents as the system encounters progressively richer data sets.
- **Recursive patterns.** Synthesis of recursive structures (fractals, parenthesisation patterns); DreamCoder learns the fixed-point combinator `Y` as an abstraction.
- **Scene reasoning, planning, and classic CS tasks.** Performance broadly matches or exceeds hand-designed program synthesis baselines.

6.2 Ablations

The most informative experimental results are not the headline accuracies but the ablations that isolate the contribution of each component:

- **Without the recognition model Q_θ .** Search is 3–10× slower; harder tasks remain unsolved within the budget. The neural recognition model is essential for tractable search at non-trivial library depth.
- **Without abstraction.** The library is fixed at its initial primitives. Performance plateaus after the first wake-sleep cycle; there is no compound learning across iterations. Library growth is what enables progressive solution of harder tasks.
- **Without fantasies.** Only replays are used to train Q_θ . The recognition model overfits to the small training set and fails to generalise to structurally new programs at test time. Fantasies are essential for generalisation of the search heuristic.

These ablations demonstrate that the three phases are not redundant; each addresses a distinct bottleneck (search efficiency, vocabulary growth, generalisation of the recognition model).

6.3 Kepler’s third law: a case study

A particularly illustrative result is the rediscovery of Kepler’s third law. Given a table of orbital radii r and periods T for the planets of the solar system, DreamCoder is asked to find a function $T = g(r)$ that fits the data. Starting from primitives $\{+, -, \times, /, \sqrt{\cdot}\}$, after several iterations on simpler regression tasks the library contains $(\cdot)^2$ and $(\cdot)^{3/2}$. The system then synthesises $T(r) = c \cdot r^{3/2}$, recovering the law as a four-token program. Without the library extensions, the same program would require ten or more tokens and would fall outside the search budget.

The point is not that DreamCoder is doing science. It is that, given a distribution of related tasks, the library captures the *notation* that makes the hard tasks tractable—in precisely the way human mathematicians build on prior vocabulary.

7 Applications to ARC-AGI

7.1 Why DreamCoder is conceptually aligned with ARC

The Abstraction and Reasoning Corpus (Chollet, 2019; Chollet et al., 2025) was explicitly designed to penalise approaches that rely on large-scale pretraining or hand-crafted DSLs. Its three defining properties—tasks too diverse to prepare for in advance, minimal prior knowledge requirements, and feasibility for untrained humans—map almost directly onto DreamCoder’s design assumptions:

- ARC requires a learner that can grow its vocabulary in response to the task distribution. DreamCoder grows the library across wake-sleep cycles by construction.
- ARC starts from minimal priors (object persistence, counting, elementary geometry). DreamCoder operates on minimal initial primitives; everything else is learned.
- ARC penalises memorisation. DreamCoder solutions are programs verified by execution; the system cannot pass ARC by memorisation in the way a TTT-based LLM can.

Chollet et al. (2025) explicitly suggests that the path forward for ARC is “deep-learning guided program search” of the kind exemplified by DreamCoder, with the engineering rigour of the best brute-force DSL solvers.

7.2 PeARL and the Bober-Irizar et al. system

Bober-Irizar and Banburski-Fahey (2024) apply a DreamCoder-style system to ARC under the name PeARL (Program Expressions for ARC-Like Reasoning). The key design choices are:

Object-centric representation. ARC grids are parsed into *objects*: connected components of same-colour cells, lines, and rectangles, each carrying attributes (colour, position, size, shape). This converts the raw pixel grid into a structured graph that the program synthesis layer operates on.

Initial DSL. The primitive set contains common grid transformations: `translate`, `rotate`, `reflect`, `recolor`, `copy`, `crop`, `fill`, `count`, `select-by-colour`, and combinators `compose`, `map-over-objects`, `filter-by-attribute`. These constitute the bootstrap library L_0 .

Wake-sleep cycles. Across the ARC training set, PeARL discovers compound transformations such as “reflect each object across its own bounding box centre” or “recolour the object adjacent to the largest object”. These become library entries that solve subsequent tasks with shorter programs.

Performance. On the ARC-AGI-1 evaluation set, PeARL achieves 18 of 400 tasks solved (4.5%). This is modest in absolute terms but illuminating:

- Every solved task comes with a human-readable program.
- The performance is competitive with GPT-4 prompted with the examples, at a small fraction of the inference cost.
- Failures are systematic and interpretable: PeARL fails on tasks that require non-object-centric reasoning (e.g. tasks where the relevant pattern is in the global texture rather than in discrete objects).

7.3 Hybrid LLM + DreamCoder approaches

A second line of work uses an LLM as the recognition model in a DreamCoder-style loop. The LLM proposes candidate programs in a typed DSL; execution feedback filters them; the abstraction phase extracts new library entries; the LLM is fine-tuned (or in-context prompted) on the new library. [Li et al. \(2024\)](#) develop one such system, BARC, combining induction (program synthesis) and transduction (direct LLM prediction) with test-time self-training. The inductive side of BARC is conceptually a DreamCoder-style abstraction loop; the transductive side is the standard test-time training used by ARC Prize winners.

7.4 The fast-versus-smart tension

A central open problem in applying DreamCoder to ARC is what [Ouellette \(2024\)](#) calls the *fast-versus-smart tension*: no deep-learning-guided program synthesis system has yet consistently outperformed a well-engineered brute-force DSL search such as the 2020 Kaggle winner ([Hodel, 2020](#)). The neural recognition network has to deliver more search-space compression than its own computational overhead costs. The analogy with chess is instructive: Stockfish, without neural networks, dominated chess for a decade before AlphaZero. The corresponding moment for ARC has not yet occurred, but DreamCoder-style systems are the most plausible candidates.

8 Connections to Course Themes

DreamCoder instantiates several of the concepts from the main course notes in a single concrete system. Table 2 maps the correspondence.

The completeness of this mapping is the principal reason DreamCoder is conceptually attractive: a single algorithmic framework instantiates many of the desirable properties that the field has identified separately.

9 Limitations and Open Problems

9.1 Limitations

Typed DSL required. Programs must be expressed in a typed functional language. For ARC, the relevant types (grids, colours, positions, objects) must be specified upfront, even if the functions over those types are learned. A more ambitious system would learn the type system itself, which is an open problem in inductive type inference.

Greedy abstraction. The abstraction phase performs greedy refactoring: at each step it adds the single best abstraction. This is a polynomial-time approximation to an NP-hard problem, and it can miss combinations of abstractions that are mutually beneficial only when introduced together. The version-space machinery of [Willsey et al. \(2021\)](#) mitigates but does not eliminate this.

Course concept	DreamCoder instantiation
Kolmogorov complexity $K(x)$	$ \varrho_x _L$ is the description length of x under library L . The MDL objective (1) is the direct operational surrogate.
Sample efficiency	A program found from 3–5 input-output examples is correct on every input. This is the strongest possible form of generalisation: the program defines a function on the entire domain.
Compositional generalisation	Library entries are reusable primitives. Solving a new task is, by construction, composing primitives in a typed way—an instance of Definition 4.3 of the main notes.
Out-of-distribution generalisation	Programs found on training tasks generalise to inputs not in any training example, because the program is a deterministic function on the entire input type, not a statistical model.
Causal reasoning	Programs are explicitly functional: the output is a deterministic function of the input through named operations. Solutions are causal by construction, not correlational.
Meta-learning	The recognition model Q_θ learns to search faster across the task distribution $p(\tau)$. This is structurally analogous to MAML (Finn et al., 2017): L plays the role of a learned prior, Q_θ plays the role of an adaptation policy.
Continual learning	The library L grows monotonically; old entries are never deleted. New tasks reuse old library entries through composition, not through parameter sharing. There is no catastrophic forgetting at the library level (Kirkpatrick et al., 2017).

Table 2: Mapping of course concepts to DreamCoder components.

Recognition model scalability. Training Q_θ on fantasies from a growing library becomes increasingly expensive as $|L|$ grows: both the token vocabulary and the typical program length increase. The fantasy distribution can also become poorly matched to the real task distribution if the library overfits to training tasks.

Semantic equivalence is not exploited. DreamCoder abstracts over *syntactic* sub-expressions: it extracts f if f literally appears as a sub-tree in many programs. It does not recognise that two syntactically different sub-trees compute the same function. Equality saturation (Willsey et al., 2021) partially addresses this, but full semantic library learning is open.

ARC-AGI-2 difficulty. Chollet et al. (2025) explicitly design ARC-AGI-2 to resist current approaches by requiring deeper multi-step compositionality and context-dependent rule application. PeARL and similar systems score in the low single digits on ARC-AGI-2.

9.2 Open problems

Neural-guided search that beats brute force. The recognition model must compress the search space more than its own computational cost. No DreamCoder-style ARC system has yet achieved this in the way AlphaZero achieved it for chess and Go.

Semantic library learning. Can the system discover that two syntactically different sub-programs are semantically equivalent and merge them? This requires equality saturation or program-equivalence reasoning at scale.

Adaptive type system. In ARC, the relevant object types (symmetry axes, connected components, bounding boxes, lines) depend on the task. Can the type system itself be learned along with the library?

Cross-domain library transfer. Can a library learned on list processing be useful for spatial reasoning? Cross-domain transfer would be strong evidence that the system has learned *genuinely general* abstractions.

Integration with language models. LLMs contain enormous amounts of programmatic knowledge but lack the correctness guarantees of execution-based program synthesis. The Wong et al. (2021) and Li et al. (2024) lines of work are early attempts to combine the two; the right interface between an LLM’s “System 1” pattern matching and DreamCoder’s “System 2” library learning is still open.

10 Conclusion

DreamCoder is a Bayesian program synthesis system that operationalises the Kolmogorov / MDL view of intelligence: it learns by compressing the descriptions of the tasks it has solved into a growing library of reusable primitives, and by training a neural recognition network to guide future search through that library. The three-phase wake-sleep algorithm performs approximate Bayesian inference over the joint structure of programs and the language they are written in.

Among the approaches surveyed for ARC-AGI, DreamCoder-style systems are the most directly aligned with what the benchmark was designed to measure: fluid intelligence as the ability to grow one’s vocabulary in response to novel tasks. The headline scores of current implementations are modest, but the conceptual fit is strong, and the trajectory from Ellis et al. (2021) through Bober-Irizar and Banburski-Fahey (2024) and Li et al. (2024) suggests that the engineering gaps are tractable. Whether a DreamCoder-style system will achieve the AlphaZero moment for ARC—decisively beating brute-force DSL search through learned search heuristics—is among the most important open questions in the field.

References

- Mikel Bober-Irizar and Andrzej Banburski-Fahey. Neural networks for abstraction and reasoning: Towards broad generalization in machines, 2024. URL <https://arxiv.org/abs/2402.03507>.
- François Chollet. On the measure of intelligence. *arXiv preprint*, 2019. URL <https://arxiv.org/abs/1911.01547>.
- François Chollet, Mike Knoop, Greg Kamradt, Bryan Landers, and Henry Pinkard. ARC-AGI-2: A new challenge for frontier AI reasoning systems. *arXiv preprint*, 2025. URL <https://arxiv.org/abs/2505.11831>.
- Peter Dayan, Geoffrey E. Hinton, Radford M. Neal, and Richard S. Zemel. The Helmholtz machine. *Neural Computation*, 7(5):889–904, 1995. doi: 10.1162/neco.1995.7.5.889. URL <https://www.cs.toronto.edu/~hinton/absps/helmholtz.pdf>.
- Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B. Tenenbaum. DreamCoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*, pages 835–850, 2021. doi: 10.1145/3453483.3454080. URL <https://arxiv.org/abs/2006.08381>.

- Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1126–1135, 2017. URL <https://arxiv.org/abs/1703.03400>.
- Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. *ACM SIGPLAN Notices*, 46(1):317–330, 2011. doi: 10.1145/1925844.1926423. URL <https://dl.acm.org/doi/10.1145/1925844.1926423>.
- Geoffrey E. Hinton, Peter Dayan, Brendan J. Frey, and Radford M. Neal. The “wake-sleep” algorithm for unsupervised neural networks. *Science*, 268(5214):1158–1161, 1995. doi: 10.1126/science.7761831. URL <https://www.science.org/doi/10.1126/science.7761831>.
- Michael Hodel. ARC 2020 solution: Combining constants and abstractions. Kaggle ARC Challenge, 1st place solution, 2020. URL <https://www.kaggle.com/c/abstraction-and-reasoning-challenge/discussion/154597>.
- Dieuwke Hupkes, Verna Dankers, Mathijs Mul, and Elia Bruni. Compositionality decomposed: How do neural networks generalise? *Journal of Artificial Intelligence Research*, 67:757–795, 2020. doi: 10.1613/jair.1.11674. URL <https://www.jair.org/index.php/jair/article/view/11674>.
- Diederik P. Kingma and Max Welling. Auto-encoding variational Bayes. In *2nd International Conference on Learning Representations (ICLR)*, 2014. URL <https://arxiv.org/abs/1312.6114>.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017. doi: 10.1073/pnas.1611835114. URL <https://www.pnas.org/doi/10.1073/pnas.1611835114>.
- Brenden M. Lake and Marco Baroni. Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018. URL <https://arxiv.org/abs/1711.00350>.
- Brenden M. Lake, Ruslan Salakhutdinov, and Joshua B. Tenenbaum. Human-level concept learning through probabilistic program induction. *Science*, 350(6266):1332–1338, 2015. doi: 10.1126/science.aab3050. URL <https://www.science.org/doi/10.1126/science.aab3050>.
- Ming Li and Paul Vitányi. *An Introduction to Kolmogorov Complexity and Its Applications*. Springer, 3rd edition, 2008. doi: 10.1007/978-0-387-49820-1. URL <https://link.springer.com/book/10.1007/978-0-387-49820-1>.
- Wen-Ding Li, Keya Hu, Carter Larsen, Yuqing Wu, Simon Alford, Caleb Woo, Spencer M. Dunn, Hao Tang, Michelangelo Naim, Dat Nguyen, Wei-Long Zheng, Zenna Tavares, Yewen Pu, and Kevin Ellis. Combining induction and transduction for abstract reasoning, 2024. URL <https://arxiv.org/abs/2411.02272>.
- Simon Ouellette. Towards efficient neurally-guided program induction for ARC-AGI. In *arXiv preprint*, 2024. URL <https://arxiv.org/abs/2411.17708>.
- Jorma Rissanen. Modeling by shortest data description. *Automatica*, 14(5):465–471, 1978. doi: 10.1016/0005-1098(78)90005-5. URL <https://www.sciencedirect.com/science/article/pii/0005109878900055>.

- Joshua B. Tenenbaum, Charles Kemp, Thomas L. Griffiths, and Noah D. Goodman. How to grow a mind: Statistics, structure, and abstraction. *Science*, 331(6022):1279–1285, 2011. doi: 10.1126/science.1192788. URL <https://www.science.org/doi/10.1126/science.1192788>.
- Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. egg: Fast and extensible equality saturation. *Proceedings of the ACM on Programming Languages*, 5(POPL):1–29, 2021. doi: 10.1145/3434304. URL <https://arxiv.org/abs/2004.03082>.
- Catherine Wong, Kevin Ellis, Joshua B. Tenenbaum, and Jacob Andreas. Leveraging language to learn program abstractions and search heuristics. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, pages 11193–11204, 2021. URL <https://arxiv.org/abs/2106.11053>.